Trabajo Práctico 6

Inciso 1

Explique brevemente los siguientes conceptos

- Parámetro
- Parámetro real
- Parámetro formal
- Ligadura posicional
- Ligadura por palabra clave o nombre

Respuesta:

- **Parámetro:** Es el mecanismo de comunicación de datos más recomendado y claro entre unidades de programa. Permite que la dependencia entre la rutina llamadora y la llamada sea explícita en la firma o interfaz de la misma, mejorando la legibilidad y facilitando el mantenimiento al reducir el acoplamiento oculto.
- **Parámetro real (o argumento):** Es el **valor, variable o expresión concreta** que se envía efectivamente al subprograma en el momento de realizar la invocación o llamada. Por ejemplo, en una llamada `max(5.0, 10.0)`, los números 5.0 y 10.0 son los parámetros reales.
- **Parámetro formal:** Es la **variable declarada en la cabecera** o prototipo de la rutina. Actúa como el receptor del dato dentro del subprograma y permite referenciar ese valor durante la ejecución de sus sentencias.
- **Ligadura posicional:** Es el método de vinculación más frecuente, donde la correspondencia entre los parámetros reales y formales se establece según su **orden físico** en la lista. Es decir, el primer parámetro real se liga al primer parámetro formal, y así sucesivamente.
- **Ligadura por palabra clave o nombre:** Es un método de vinculación donde el parámetro real se asocia directamente mediante el **nombre explícito del parámetro formal** (por ejemplo, `area(base=10, altura=5)`). Este mecanismo ofrece mayor flexibilidad al permitir cambiar el orden de los argumentos en la llamada y mejorar la documentación, aunque requiere que el programador conozca los nombres internos definidos en la rutina

Inciso 2

Unir los siguientes puntos según corresponda y de una definición y un ejemplo
de cada par.

	Resultado
	Valor
Modo IN	Valor / Resultado
Modo OUT	Nombre
modo IN / OUT	Resultado de funciones
	Valor Constante
	Referencia

1. Metodo IN (Entrada)

En este modo, el flujo de información es unidireccional hacia la rutina llamada.

- **Valor:**
  - **Definición:** Se realiza una **copia física** del parámetro real en el parámetro formal; cualquier cambio realizado dentro de la rutina no afecta a la variable original.
  - **Ejemplo:** En el lenguaje C, al pasar un entero a una función `void inc(int v)`, si se invoca con `inc(a)` y `a` vale 10, tras la ejecución `a` seguirá valiendo 10 aunque el formal se incremente.
- **Valor Constante:**
  - **Definición:** Es similar al pasaje por valor (se recibe una copia), pero el lenguaje **prohíbe explícitamente** que la rutina modifique el parámetro formal.
  - **Ejemplo:** El uso de la palabra reservada `const` en los parámetros de C++ para asegurar que el dato no sea alterado por accidente.

2. Metodo OUT (Salida)

El flujo de información es unidireccional hacia la rutina llamadora una vez que la ejecución finaliza.

- **Resultado:**
  - **Definición:** El parámetro formal **no tiene un valor inicial** al entrar a la rutina; al terminar la ejecución, su contenido se copia de vuelta a la dirección de memoria (l-valor) del parámetro real.
  - **Ejemplo:** Un procedimiento que calcula un código de estado y lo "devuelve" a través de un parámetro que el invocador dejó vacío.
- **Resultado de funciones:**
  - **Definición:** Es el valor explícito que una **función** devuelve como resultado final al punto donde fue invocada dentro de una expresión.
  - **Ejemplo:** Una función `factorial(n)` que, tras realizar sus cálculos, utiliza una sentencia `return` para entregar el r-valor final al programa principal.

3. Metodo IN/OUT (Bidireccional)

Permite que la información fluya en ambos sentidos entre la unidad llamadora y la llamada.

- **Resultado (referido a Valor-Resultado):**
  - **Definición:** Combina los dos momentos: **copia el valor al entrar** a la rutina y **vuelve a copiar el resultado** al parámetro real al salir; la rutina trabaja con una copia local durante su ejecución.
  - **Ejemplo:** Una rutina en la que el parámetro real se actualiza solo en el instante en que la subrutina finaliza su ejecución (copia y restauración).
- **Referencia:**
  - **Definición:** Se pasa la **dirección de memoria (l-valor)** del parámetro real; la rutina opera directamente sobre la variable original, por lo que los cambios son inmediatos.
  - **Ejemplo:** En C++, declarar un parámetro formal anteponiendo el símbolo `&` (como en `void inc(int &v)`), lo que permite que la variable original del invocador sea modificada directamente.
- **Nombre:**
  - **Definición:** El parámetro formal se **reemplaza textualmente** por el parámetro real (evaluación perezosa); la expresión se evalúa de nuevo

cada vez que el nombre aparece en el código de la rutina.
- **Ejemplo:** La implementación mediante **Thunks** en lenguajes como Algol, donde el valor del parámetro puede cambiar si las variables de la expresión original se modifican durante la ejecución de la rutina.

Inciso 3

1. Complete el siguiente cuadro según lo correspondiente a cada lenguaje

Tipo de pasaje de parámetros	Lenguaje
	ADA
	C
	Rubi
	JAVA
	Python

2. Ada es más seguro que Pascal, respecto al pasaje de parámetros en las funciones. Explique por qué.
3. Explique cómo maneja Ada los tipos de parámetros in-out de acuerdo al tipo de dato

Inciso 3.1

Tipo de pasaje de parámetros	Lenguaje
In, Out, In Out	ADA
Por valor (punteros para simular referencia)	C
Por valor de referencia (o por objeto)	Rubi
Por valor (las referencias se pasan por valor)	JAVA
Por asignación (o por objeto/referencia)	Python

Inciso 3.2

La mayor seguridad de **Ada** respecto a **Pascal** en el pasaje de parámetros dentro de las **funciones** radica en la restricción de los **efectos colaterales**:

- **Ada:** Por diseño, los parámetros de las funciones en Ada solo pueden ser de **modo IN** (entrada). Esto garantiza que las funciones sean "puras" (similares a las funciones matemáticas), impidiendo que la rutina modifique los argumentos originales y asegurando que solo devuelvan un valor explícito como resultado.
- **Pascal:** En contraste, Pascal permite el uso de parámetros **var** (pasaje por referencia) dentro de las funciones. Esto permite que una función modifique sus parámetros de entrada, provocando efectos colaterales que dificultan la depuración y reducen la confiabilidad del código al permitir cambios inesperados en el estado del programa,.

Inciso 3.3

Ada gestiona los parámetros de modo **IN OUT** (bidireccional) utilizando dos estrategias diferentes dependiendo de la naturaleza del dato para balancear eficiencia y seguridad:

- **Tipos Escalares o Simples:** Se manejan mediante **Valor-Resultado** (**Copy-in**/**Copy-out**). El sistema realiza una copia física del valor al entrar a la rutina y vuelve a copiar el valor final al parámetro real solo cuando la rutina termina. Esto protege al parámetro real de cambios intermedios si la rutina falla.
- **Tipos Compuestos o Grandes** (**Arreglos y Registros**)**:** Se manejan generalmente por **Referencia**. En lugar de copiar toda la estructura (lo cual sería costoso en tiempo y memoria), se pasa la dirección de memoria (l-valor) del parámetro real,. Esto permite que la rutina opere directamente sobre la estructura original, maximizando la eficiencia de ejecución.

Inciso 4

Sea el siguiente programa escrito en Pascal-like

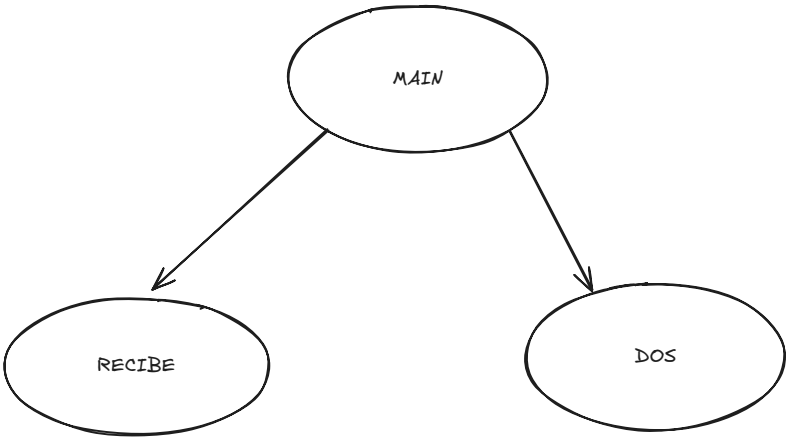
```
Procedure Main;
var j, m, i: integer;

Procedure Recibe (x:integer; y:integer);
        begin
                m:= m + 1 + y;
                x:=i + x + j;
                y:=m - 1;
                write (x, y, i, j, m);
        end;
```

```
Procedure Dos;
        var m:integer;
        begin
                m:= 5;
                Recibe(i, j);
                write (i, j, m);
        end;
begin
        m:= 2;
        i:=1; j:=3;
        Dos;
        write (i, j, m);
end.
```

1. Arme el árbol de anidamiento sintáctico y el registro de activación de cada una de las unidades.
2. Decir qué imprime el programa suponiendo que para todas las variables que se pasan el pasaje de parámetros es por: (Deberá hacer la pila estática y dinámica para cada caso)
    1. Referencia.
    2. Valor
    3. Valor Resultado
    4. Nombre
    5. Resultado.
3. ¿Existió algún caso que no pudo realizarlo porque saltó algún tipo de error? Diga cuál y por qué.
4. ¿Dará el mismo resultado si se trata de un lenguaje que sigue la cadena dinámica? Justifique la respuesta realizando las pilas de activación

Inciso 4.1



Registros de activación
Reg activación main
PR
LE
LD
J
M
I
Procedure Recibe
Procedure Dos
VR

Registros de activación
Reg activación recibe
PR
LE
LD
param x
param y
VR

Registros de activación
Reg activación Dos
PR
LE
LD
m
VR

Inciso 4.2

Por Referencia

Cadena estática:

Pila de ejecución

Registros de activación *1
Reg activación main
PR
LE
LD
J ~~3~~ 5
m ~~2~~ 6
i ~~1~~ 5
Proc recibe
Proc dos
VR

Registros de activación *2
Reg activación Dos
PR
LE (*1)
LD (*1)
m 5
VR

Registros de activación *3
Reg activación recibe
PR
LE (*1)
LD (*2)
x ^i
y ^j
VR

Impresiones/operaciones adicionales

Reg Activación:

- *1:
  - Imprime "5", "5", "6"
- *2:
  - Imprime "5", "5", "5"
- *3:
  - m = m + 1 + y = 2 + 1 + 3 = 6
  - x = i + x + j = 1 + 1 + 3 = 5
  - y = m - 1 = 6 - 1 = 5
  - Imprime "5", "5", "5", "5", "6"

Inciso 4.3

Existió algún caso que no pudo realizarlo porque saltó algún tipo de error? Diga cuál y por qué.

En el caso de la resultado no se puede realizar ni por cadena dinámica ni estática, debido a que al querer ejecutar la sentencia , al no estar inicializado y, falla el programa.

Inciso 4.4

¿Dará el mismo resultado si se trata de un lenguaje que sigue la cadena dinámica?
Justifique la respuesta realizando las pilas de activación

Si, da lo mismo porque la sentencia es independiente de que cadena utilicemos.

Inciso 5

Suponiendo que se está ejecutando un programa con el siguiente registro de activación en memoria y se llama al procedimiento rutina(iter,vec,a). Determine el tipo de parámetro que se deben utilizar en el llamado para que los resultados sean los siguientes:

a) (4,6,7),(4,6,7), 2, 2 Referencia los 3 parametros
b) (3,5,6),(4,6,7), 2, 2 por valor la variable vector y referencia vit e iteracion
c) (3,5,6),(5,5,6), 0, -1 por valor o valor resultado

PR
LD
LE
Iter: true
Vec:[3,5,6]
a: -1
Rutina()
VR

```
......
procedura rutina(tipoParam iteracion,tipoParam
vector,tipoParam vit):
```

```
while iteracion begin

    vit = a+1

    vector[vit] = vector[vit]+1

    iteracion = (vector[vit] mod 2)==0

end

print vec

print vector

print vit

print a
.....
rutina(iter,vec,a)
```

Inciso 6

Indique con un ejemplo el comportamiento del parámetro por nombre (en el parámetro formal) para los siguientes casos de parámetros reales:

- Un valor entero.
- Una constante.
- Un elemento de un arreglo.
- una expresión.

Que sucede en cada caso?

A continuación, se detalla el comportamiento para cada caso solicitado:

1. Un valor entero (literal)

- **Qué sucede:** El compilador o intérprete intenta reemplazar el nombre del parámetro formal por el número literal (por ejemplo, `5`).
- **Comportamiento:** Si la rutina solo **lee** el parámetro, funciona correctamente devolviendo el valor. Sin embargo, si la rutina intenta **asignarle** un nuevo valor (ej. `x := x + 1`), se producirá un **error**

semántico. Esto se debe a que un literal no es un **l-valor** (no tiene una dirección de memoria donde se pueda escribir) y la sentencia resultante sería inválida (ej. `5 := 5 + 1`).

2. Una constante

- **Qué sucede:** Similar al valor entero, el nombre de la constante sustituye al parámetro formal.
- **Comportamiento:** Si se intenta modificar el parámetro dentro de la rutina, el programa fallará. Dado que las constantes son **inmutables** y su ligadura r-valor es fija, no pueden actuar como receptores de una asignación destructiva. Si el lenguaje maneja constantes por sustitución directa en el código, el error será similar al de intentar modificar un número literal.

3. Un elemento de un arreglo

- **Qué sucede:** Se sustituye el formal por la referencia indexada, por ejemplo `A[i]`.
- **Comportamiento:** Es el caso más distintivo. Si la variable índice (`i`) cambia su valor dentro de la rutina (ya sea porque es una variable global o porque se modificó a través de otro parámetro), la referencia al parámetro formal **cambiará de celda de memoria** en cada acceso.
- **Ejemplo:** Si se accede a `x`, luego se incrementa `i`, y se vuelve a acceder a `x`, se estarán leyendo o modificando elementos diferentes del arreglo (`A` y luego `A`) dentro de la misma ejecución de la rutina.

4. Una expresión

- **Qué sucede:** El parámetro formal se reemplaza por la expresión completa, como `(a + b)`.
- **Comportamiento:** La expresión se **reevalúa por completo** cada vez que el parámetro es referenciado. Si los componentes de la expresión (`a` o `b`) son variables que cambian durante la ejecución de la rutina, el "valor" del parámetro formal será distinto en cada momento.
- **Ejemplo:** Si la expresión es `base * altura` y la rutina modifica la variable `base` a mitad de su ejecución, la siguiente vez que use el parámetro formal obtendrá un resultado actualizado basado en el nuevo valor de `base`.

Inciso 8

1. Indique las diferencias entre los pasaje de subprogramas como parámetros deep y shallow.
2. Realice la pila estática y dinámica tanto con el pasaje de parámetros deep y shallow para el siguiente código.

```
Program A
 Var x:integer;
 Var y: char;
 Procedure B;
    Var h:integer;
    Begin
      h:=1+x;
      Write (y);
      C(D);
      Write (y);
    End;
  Procedure C (Subrutina S);
     Var x:integer;
     Var y: char;
     Begin
       x:=3;
       y:= "b";
       x:=S(x,y)
       y:= "j";
       Write (x,y);
     End;
```

```
 Function D (j:integer, k:char);
   Begin
      j:=j+x;
      k:=y;
      Write (k);
      Return j;
    End;
BEGIN
 x:=0;
 y:="a";
 B();
 Write (x,y);
END.
```

Inciso 8.1

Cuando un subprograma se pasa como parámetro a otra rutina, surge el problema de determinar cuál es el **ambiente de referencia** que debe utilizarse para resolver sus variables no locales. Las diferencias entre los métodos **deep** (profundo) y **shallow** (superficial) radican precisamente en qué momento o lugar de la ejecución se fija ese ambiente:

- **Ligadura Profunda (Deep Binding):**
  - **Ambiente de referencia:** Es aquel donde el subprograma fue **definido** originalmente.
  - **Contexto de uso:** Es el estándar para lenguajes que utilizan **alcance estático**.
  - **Justificación:** Garantiza la coherencia lógica, ya que la función siempre accede a las variables que "veía" en el momento en que fue escrita, sin importar desde qué parte del programa sea invocada finalmente.
- **Ligadura Superficial (Shallow Binding):**
  - **Ambiente de referencia:** Es aquel que **invoca** al subprograma en tiempo de ejecución.

- **Contexto de uso:** Es típico de lenguajes que operan con **alcance dinámico**.
- **Comportamiento:** Las variables no locales se resuelven buscando en la cadena de ejecución (quién llamó a quién) en el momento exacto de la llamada.

Inciso 9

Sea el siguiente código escrito en Pascal like

```
Procedure main
        a: array(1..5) of integer;
        x: integer;
        i;integer;
        Procedure Uno (tipo_pasaje
        m:integer)
                Begin
                x:=0;
                x:=x+1;
                m:=m+x + a(3);
                x:=x*2;
                a(3):=a(3) - 1;
                m:=m+1;
                End;
```

```
Begin
For i:=1 to 5 a(i):=1;
x:=3;
Uno(a(x));
For i:=1 to 5 write (a(i));
End.
```

1. Plantee diferencias, relacionada con la forma de implementación de cada uno y los resultados sobre este ejemplo considerando los siguientes tipos de pasajes parámetros nombre, referencia y valor resultado.
2. ¿Qué sucede si en Uno se agrega la siguiente declaración: x: integer? Indique el resultado para cada uno de los tipos de pasajes de parámetros (nombre, referencia y valor resultado)

Inciso 9.1

a) Diferencias en implementación y resultados

A continuación se detallan las diferencias operacionales y los resultados finales de la impresión del arreglo `a` (inicialmente todos en 1 y `x = 3`):

1. Pasaje por Referencia

- **Implementación:** Se pasa la **dirección de memoria** (**l-valor**) del parámetro real `a(x)`. El parámetro formal `m` se convierte en un **alias** de `a(3)`. Cualquier cambio en `m` es una modificación inmediata en la celda original.
- **Ejecución:**
  1. `x := 0` y `x := 1` modifican el `x` global.

    2. `m := m + x + a(3)` → `a(3) := 1 + 1 + 1 = 3`.
    3. `x := 2`.
    4. `a(3) := a(3) - 1` → `a(3) := 3 - 1 = 2`.
    5. `m := m + 1` → `a(3) := 2 + 1 = 3`.
  - **Resultado (Imprime): 1, 1, 3, 1, 1**

2. Pasaje por Valor-Resultado (Copy-in/Copy-out)

  - **Implementación:** Se realiza una **copia física** del valor al entrar a la rutina (`copy-in`) y se vuelve a copiar el resultado final al parámetro real al terminar (`copy-out`). No hay vínculo entre `m` y `a(3)` durante la ejecución.
  - **Ejecución:**
    1. `m` recibe el valor 1.
    2. `x := 1` (global).
    3. `m := 1 + 1 + a(3)` → `m = 3` (local).
    4. `a(3) := a(3) - 1` → El `a(3)` global pasa a ser 0.
    5. `m := 3 + 1 = 4`.
    6. Al finalizar, `m` (4) se copia en `a(3)`.
  - **Resultado (Imprime): 1, 1, 4, 1, 1**

3. Pasaje por Nombre (Lazy Evaluation)

  - **Implementación:** Se basa en la **sustitución textual** del parámetro formal por el real en cada punto de uso. Se implementa mediante un **Thunk**, evaluando la expresión `a(x)` cada vez que aparece `m`.
  - **Ejecución:**
    1. `x := 1` (global).
    2. `m := m + x + a(3)` → `a(x) := a(1) + 1 + a(3)` → `a(1) := 1 + 1 + 1 = 3`.
    3. `x := 2`.
    4. `a(3) := a(3) - 1` → `a(3) := 1 - 1 = 0`.
    5. `m := m + 1` → `a(x) := a(2) + 1` → `a(2) := 1 + 1 = 2`.
  - **Resultado (Imprime): 3, 2, 0, 1, 1**

Inciso 9.2

Si se agrega `x: integer` dentro de `Uno`, se produce un **enmascaramiento** (**shadowing**): las referencias a `x` dentro de la rutina se resolverán en el RA local de `Uno` y no afectarán al `x` global del `main`.

1. Referencia

- **Comportamiento:** `m` se liga al l-valor de `a(3)` en el momento de la llamada (donde el `x` global valía 3). Los cambios al `x` local no afectan esta ligadura.
- **Resultado:** Idéntico al caso (a) para el arreglo: **1, 1, 3, 1, 1**. El `x` global permanece en 3.

2. Valor-Resultado

- **Comportamiento:** `m` toma el valor 1. El proceso interno usa el `x` local, pero como `m` es una copia independiente, el resultado local de `m` no cambia. Al final, se restaura sobre `a(3)`.
- **Resultado:** Idéntico al caso (a) para el arreglo: **1, 1, 4, 1, 1**. El `x` global permanece en 3.

3. Nombre

- **Comportamiento:** Por regla de **ligadura profunda** (**deep binding**), el Thunk de `a(x)` debe resolver `x` en el ambiente donde se realizó la llamada o donde se definió, para mantener la coherencia del alcance estático. Por lo tanto, `m` siempre referenciará a `a(global x)`, que es `a(3)`.
- **Ejecución:** Aunque el `x` local de `Uno` cambia, `m` siempre actúa sobre `a(3)`.
  1. `m := a(3) + x_local(1) + a(3)` → `a(3) = 1 + 1 + 1 = 3`.
  2. `a(3)` global cambia a 2 (instrucción `a(3) := a(3) - 1`).
  3. `m := a(3) + 1` → `a(3) = 2 + 1 = 3`.
- **Resultado** (**Imprime**): **1, 1, 3, 1, 1**. El `x` global permanece en 3.

Inciso 10

Sea el siguiente un programa escrito en Pascal:

```
Program Uno;
        var x:integer;
        Function Dos:integer;
                begin
                x:= x + 1;
                return (x);
                end;
```

```
        Procedure Tres (pasaje x:integer);
                begin
                x:= x + 5;
                x:= Dos + 10;
                end;
begin
x:= 8; Tres(x);
write (x);
end.
```

1. Explique cómo simularía en Pascal el pasaje por valor-resultado y hágalo sobre este ejemplo.
   Nota: No se pueden agregar más variables, ni cambiar el nombre de las que están.
2. Transcriba este ejemplo en Ada de manera tal que el resultado de la ejecución sea diferente si el pasaje de parámetros es por referencia y luego por valor – resultado